

自然语言处理

课程设计

Lab 1

中文纠错

院系：人工智能学院

姓名：石睿

学号：211300024

班级：自然语言处理-2025 春季学期

邮箱：211300024@smail.nju.edu.cn

实验时间：2025.4.13

一、 实验进度

我已经完成了课程设计 1：中文纠错中的所有内容，包括数据分析、基于规则和基于统计的方法，并在统计方法中引入了基于深度学习的方法。

额外添加的 bash 文件说明：

已创建了一个 bash 文件 (command.sh)，其中包含了每一个实验所需的运行命令（已被注释掉）。若您的运行环境是基于 Linux 的，您可以直接在终端输入 `bash command.sh` 来完成 pipeline 的运行。如若是 window 环境，您可以直接复制 command.sh 中的脚本代码并在终端运行。同时我也会在下方每一个细节说明中给出该方法应该使用哪种命令来运行，以方便助教哥哥/姐姐进行复现。

额外添加的实验文件说明 – 均在 exp 文件夹中

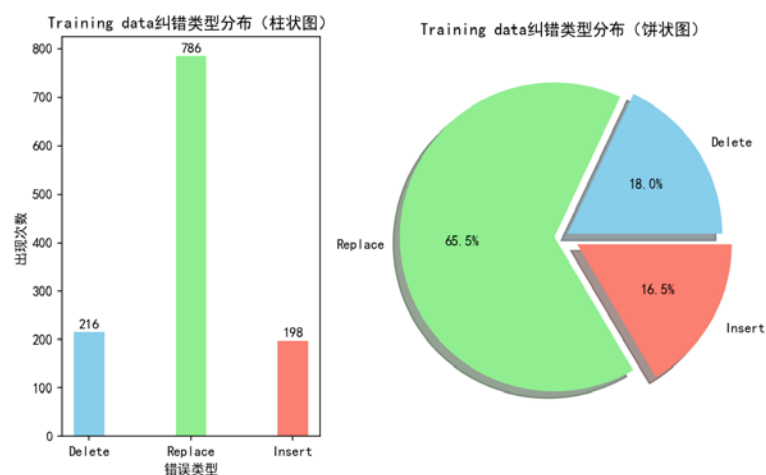
基于规则的方法由于主代码部分改动不大，故所有不是最优效果的实现放到了 `exp_rule_based.py` 中。助教哥哥/姐姐如若尝试，可以把其中的函数复制到 `rule_based.py` 中运行。

基于统计的方法由于不同方法之间差距比较大，故保存了每一个完成的 python 文件，包括：`exp_statistical_bert_LSTM.py`（也是 `statistical.py` 中的内容啦），`exp_statistical_thidf_crf.py`，`exp_statistical_thidf_logistic.py` 三个文件。如若尝试，可以把整个文件内容复制到 `statistical.py` 中运行。

二、 实验结果

以下是各个任务的实现的最优效果简述，具体细节将在第三部分阐述。

2.1 数据分析



2.2 规则方法

```
Sample-level Evaluation:
Accuracy: 0.4924
Character Accuracy: 0.9039

Detection Evaluation:
Precision: 0.4409
Recall: 0.0743
F1 Score: 0.1271
F0.5 Score: 0.2219

Correction Evaluation:
Precision: 0.4318
Recall: 0.0729
F1 Score: 0.1247
F0.5 Score: 0.2175

Final Score:
F0.5 Score: 0.2175
```

2.3 统计方法 (数据太少啦, 我尽力只能做到这个效果啦 o(╥_╥)o)

```
Sample-level Evaluation:
Accuracy: 0.5682
Character Accuracy: 0.8712

Detection Evaluation:
Precision: 0.7288
Recall: 0.6172
F1 Score: 0.6683
F0.5 Score: 0.7033

Correction Evaluation:
Precision: 0.3626
Recall: 0.4451
F1 Score: 0.3996
F0.5 Score: 0.3765

Final Score:
F0.5 Score: 0.3765
```

三、 实验细节

3.1 数据分析

analyze_data 函数：如 ppt 中所述，“纠错任务”从语义上可以分为 spelling errors, redundant words, missing words, word order errors, 但这种分类对语义强依赖。利用 Levenshtein 库函数把纠错行为分类成了“替换”，“删除”，“插入”三个子任务的识别，并进行统计。

visualize_error_distribution：利用 matplotlib 绘制了柱状图（统计数量），和饼图（直观显示各类别占比），如“实验结果”中所述。

3.2 规则方法

由于项目细节过多，以下只重点表述重要细节和思考。

3.2.1 Train:

_extract_confusion_pairs：框架代码已实现，但具体实验中发现只基于词频统计的方式会极大扰乱原有仅有少量错误的文本，故在 train 和 correct 函数中注释掉了这个方法。

_extract_punctuation_rules：为了训练数据量充足的考虑，我并没有引入 `len(source) == len(target)` 这个限制。同时我发现不能只做位置对比，如果 target 在 source 的基础上发生了删除/插入，source 和 target 发生错位，匹配统计出来没有任何意义。故在该函数中也使用了 Levenshtein 分析编辑操作，并只筛选“replace”找到并“学习”标点符号的错误替换。该函数中，我定义了一组常用的中文标点符号，将对这些标点进行识别和学习 `punctuation_chars = set(",。! ? ; : () 【】《》“”‘’")`

_extract_grammar_rules 函数的三种方法：

- 基于词性（POS）的语法模式抽取方法【**最优，当前 rule_based.py 中的代码**】

```
python main.py --method 'rule_pos' --analyze --evaluate
```

F0.5 Score: **0.2175**。使用 jieba.posseg 中的 cut 函数得到（词，词性）的 pairs，尝试

从训练数据中“学习”到（统计出）(s_char, left_pos, mid_pos, right_pos) → t_char 形式的规则，其中 s_char 是 source 中出错的字符，t_char 是 target 中正确的字符，left_pos/mid_pos/right_pos 分别是该字符左右的词性上下文。由于这样学习存在很强的 bias（只通过左右词性推断中间词汇的词性），我设置了替换频次 ≥ 10 次才计入计算，同时保证了上下文窗口大小为 2。学习到的语法结构为：{('的', 'd', 'u', 'v'): {'地': N1 }, ('地', 'u', 'd', 'v'): {'的': N2 }, ('再', 'v', 'd', 'v'): {'在': N3 }}, 其中 N1, N2, N3 分别为出现的次数数量。

```
[Config -] context length: 2,count limit: 10
{('的', 'a', 'uj', 'v'): {'地': 45}, ('的', 'd', 'uj', 'v'): {'地': 35}, ('的', 'i', 'uj', 'v'): {'地': 12}, ('的', 'ad', 'uj', 'v'): {'地': 11}}
```

```
Sample-level Evaluation:
Accuracy: 0.4924
Character Accuracy: 0.9039

Detection Evaluation:
Precision: 0.4409
Recall: 0.0743
F1 Score: 0.1271
F0.5 Score: 0.2219

Correction Evaluation:
Precision: 0.4318
Recall: 0.0729
F1 Score: 0.1247
F0.5 Score: 0.2175

Final Score:
F0.5 Score: 0.2175
```

- 成分句法分析

因为成分句法分析已被舍弃，同时也不在 huggingface 或 ltp 中被支持，故未得到结果。我尝试使用 ltp 库来实现成分句法分析，来获得每个 token 的 constituency。我尝试使用 pipeline 进行分词（当前 ltp 不再支持 seg 和 encoder 接口），但由于 tasks 中不再有实现 constituent 成分分析的任务，故未继续实现。后续也尝试了 huggingface 中的模型进行成分句法分析，但我没有找到能实现这一功能的专门模型，故也未继续实现。尝试的代码在 rule_based_experients.py 中，供助教哥哥/姐姐查看。

- 依存句法分析

```
python main.py --method 'rule_dep' --analyze --evaluate
```

Tip: 可以通过 main.py 的接口运行，也可以直接运行 rule_based.py，已经设立了一个 if __name__ == "__main__" 接口用于测试

F0.5 Score: **0.0059**。LTP 中 tasks 中支持"dep"，故依存句法分析可以实现。我定义了一个 LTPDependencyParser 类，初始化了"LTP/base"模型（也可以选择更大的模型实现效果会更好）

如果仿照给出的函数，也即只做 $\text{len}(\text{src}) == \text{len}(\text{tgt})$ 的 replace 操作的检测和学习，而不使用 Levenshtein。但是在替换频次 ≥ 5 的筛选条件下，学习出的 dependency grammar 二者相差无几。但从理论上，使用 Levenshtein 就不用限制 $\text{len}(\text{src}) == \text{len}(\text{tgt})$ 啦，能学到更多更准确的 grammar rules 呢。

```
[Dependency Grammar] 规则数量: 8
{('的', 'RAD'): {'地': 220, '得': 76}, ('象', 'ADV'): {'像': 7}, ('地', 'RAD'): {'的': 5}, ('在', 'ADV'): {'再': 8}, ('的', 'ADV'): {'地': 10}, ('做', 'COO'): {'作': 6}, ('做', 'HED'): {'作': 5}, ('它', 'ATT'): {'他': 7}}
```

未使用 Levenshtein，只做 $\text{len}(\text{src}) == \text{len}(\text{tgt})$ 的 replace 操作的检测和学习

```
[Dependency Grammar] 规则数量: 9
{('的', 'RAD'): {'地': 225, '得': 78}, ('象', 'ADV'): {'像': 7}, ('地', 'RAD'): {'的': 5}, ('在', 'ADV'): {'再': 9}, ('的', 'ADV'): {'地': 10}, ('做', 'COO'): {'作': 6}, ('做', 'HED'): {'作': 5}, ('它', 'ATT'): {'他': 7}, ('做', 'VOB'): {'作': 5}}
```

使用 Levenshtein，可以学习不同长度的 string 的 grammar rules

请注意，依存分析是对现有词/字符的结构功能标签进行注释，我们要分析的文本是句法完整、词语位置已定的句子。而 delete 和 insert 操作，被"删除"或被"插入"的字符或词，在原句中并不存在，无法为其获取依存标签。故此时限定为"replace"操作。这也是 rule-based 方法效果不佳的原因，具体详细的分析会在文档第四部分进行分析。

```
Sample-level Evaluation:
Accuracy: 0.1432
Character Accuracy: 0.8634

Detection Evaluation:
Precision: 0.0636
Recall: 0.2902
F1 Score: 0.1043
F0.5 Score: 0.0753

Correction Evaluation:
Precision: 0.0490
Recall: 0.2395
F1 Score: 0.0813
F0.5 Score: 0.0582

Final Score:
F0.5 Score: 0.0582
```

使用依存句法分析时效果不佳

_extract_word_confusion: 这个和 confusion_pairs 的区别是进行了 jieba 分词，将基于

word-level 做统计。同上面分析，也使用 Levenshtein，并限定搭配'replace'操作，来做 word-level 的混淆对齐。

3.2.2 Correct:

Train 中各函数写出来了，correct 中函数逻辑就是类似的啦。我在 _correct_confusion_chars 中也手动添加了一些替换规则，但是实验证明没有太多的用呢~~

3.3 统计方法

n-gram:

```
python main.py --method 'statistical' --statistical_method 'ngram' --analyze --evaluate
```

_train_ngram_model: 通过了上下文窗口，统计了各个 unigram, bigram, trigram, four-gram 的出现频率。随后统计了上下文大小为 4 的 Confusion matrix 和错误概率，并进行归一化。

_correct_with_ngram: 给定一条待纠正的文本 text 时，算法对每个位置 i（字符 char=text[i]）按以下顺序尝试：① 基于上下文 Confusion matrix 纠错 ② 上下文不匹配，看基于“错字”本身的通用 Confusion matrix 的统计纠错。③ n-gram 做把不同 n 取值的插值评分，该字出错概率较高（error_probs[char]>=0.05）且前后均有上下文时，才启动 N-gram 评分并替换。

基于 sklearn 库的机器学习模型:

```
python main.py --method 'statistical' --statistical_method 'ml' --analyze --evaluate
```

TF-IDF 做特征提取, LogisticRegression 做 detector 和 corrector

根据代码框架中的提示，我尝试了 TfidfVectorizer 作为文本特征提取器，和 LogisticRegression 作为 detector 和 corrector。因为 CountVectorizer 只统计次数，适合在分

类器里再做标准化/缩放。但是 TfidfVectorizer 不仅统计次数，还除以该 token 的文档频率 (DF)，降低高频 token 的权重，突显那些“区分度高”的 token。故该实验中未尝试 CountVectorizer。

该任务的核心思想是把纠错任务看成多分类问题，通过上下文信息预测中心词，不论这个中心词是正确的还是错误的，也即 x: (左 2, 中心, 右 2), y: target 中心词。该任务对超参数极其敏感，经过多次调参尝试，我把 TF-IDF 的最大特征数设置为 70, detector 和 corrector 的迭代训练次数均设置为 1000。

实验结果如下，在后续完成基于预训练 Bert 模型完成任务时，我发现这样设计的几种缺点：① 未经训练的 tf-idf 做 context 的 embedding 未必（大概率不适用）中文纠错任务 ② 这里 detector 只做了判断某个字符是否是正确的（二分类问题），搭配后续 corrector 做 target 的预测，这样的设定其实只完成了“替换错误”这一种错误形式。尽管'replace'这种错误在三种错误中占比最多，但该设计仍具有先天的缺陷性。

```
Sample-level Evaluation:
Accuracy: 0.2051
Character Accuracy: 0.8435

Detection Evaluation:
Precision: 0.0087
Recall: 0.0222
F1 Score: 0.0125
F0.5 Score: 0.0099

Correction Evaluation:
Precision: 0.0069
Recall: 0.0177
F1 Score: 0.0099
F0.5 Score: 0.0079

Final Score:
F0.5 Score: 0.0079
```

TF-IDF, CRF

我也尝试了 CRF 的判别式建模，并在 StatisticalCorrector 写了一个类方法，从上下文的 context list 中转化为 CRF 特征的 dict 序列。

但由于 CRF 的序列化信息来建模，保留的上下文窗口为 5 时的信息不足以让 CRF 学到很好。同时在上文中提到的两种设计缺陷在这里并为做出改善，故效果不佳


```
Sample-level Evaluation:
Accuracy: 0.4682
Character Accuracy: 0.9039

Detection Evaluation:
Precision: 0.0333
Recall: 0.0008
F1 Score: 0.0015
F0.5 Score: 0.0035

Correction Evaluation:
Precision: 0.0333
Recall: 0.0008
F1 Score: 0.0015
F0.5 Score: 0.0035

Final Score:
F0.5 Score: 0.0035
```

基于 Pytorch 的深度学习模型：

```
python main.py --method 'pretrained' --analyze --evaluate
```

Bert+BiLSTM

	模型架构设计
Bert	冻结所有的参数
Detector	BiLSTM，把 Bert 的 embedding 投射到 4 个纠错类别：保持、替换、删除、插入
Corrector	BiLSTM，把 Bert 的 embedding 投射到 Bert 的 vocabulary 中
Trick	① 【错误】考虑到数据量很小，这里还对 Detector 和 Corrector 的 BiLSTM 的特征识别层做了网络共享，只对 BiLSTM 后的 linear 增加了两个做不同任务的 head ② 【正确】四个纠错类别样本数量极度不平衡 {0: 51392, 1: 396, 2: 161, 3: 93}，对替换、删除、插入类别做了过采样。

以下结果是在训练集上训了 5 个 epochs，验证集划分为训练集的 10%，在终端打印训练结果时发现了轻微的过拟合现象（但不是很严重）

```

Sample-level Evaluation:
Accuracy: 0.1203
Character Accuracy: 0.6323

Detection Evaluation:
Precision: 0.0206
Recall: 0.1126
F1 Score: 0.0348
F0.5 Score: 0.0246

Correction Evaluation:
Precision: 0.0157
Recall: 0.0881
F1 Score: 0.0266
F0.5 Score: 0.0188

Final Score:
F0.5 Score: 0.0188

```

Bert+BiLSTM+CRF

	模型架构设计
Bert	不冻结任何的参数
Detector	linear layer + CRF, 把 Bert 的 embedding 投射到 4 个纠错类别: 保持、替换、删除、插入
Corrector	2*MLP (linear + layernorm + GELU + Dropout) + BiLSTM + Linear, 把 Bert 的 embedding 投射到 Bert 的 vocabulary 中。
Trick	① 【正确】 Detector 和 Corrector 单独训练 (除了拿到相同的 Bert embedding, 不共享任何网络结构), 并在最后 loss backward 时做了 loss 加权 (因为我发现 detector 往往训练的好, corrector 往往训练的不好)。 ② 【正确】 四个纠错类别样本数量极度不平衡, 对替换、删除、插入类别做了过采样

我原本以为这样极小的训练数据集无法在不冻结 Bert 的参数时得到很好的训练效果, 但是在读了 <https://aclanthology.org/2020.acl-main.82.pdf> 同样也是在做中文纠错这篇文章后发现 Bert 不进行微调和进行微调的性能差距巨大。同时由于训练数据集大小过小 (当前任务只有 2000 多个), 性能自然无法和文章中表格列出的至少 500,000 个样本对 Bert 模型进行微调的效果进行对比啦。

我认为在当下极小数据集训练模型, 最好的方法是做一个 multi-task 的任务设定,

Bert+Detector 和 Bert+Corrector 都各自训练, 在提高训练数据量之后可以做 end-to-end 的训练。但受限于计算资源, 这里就共享了 Bert 的 embedding, 并根据 detector 和 corrector 的 loss 对 Bert 做一次微调。

```
Sample-level Evaluation:
Accuracy: 0.5682
Character Accuracy: 0.8712

Detection Evaluation:
Precision: 0.7288
Recall: 0.6172
F1 Score: 0.6683
F0.5 Score: 0.7033

Correction Evaluation:
Precision: 0.3626
Recall: 0.4451
F1 Score: 0.3996
F0.5 Score: 0.3765

Final Score:
F0.5 Score: 0.3765
```

四、 BUG 与思考

BUG: 使用 try exception 结构时, exception 的处理要写好, 不能直接 continue, 这样 try 尝试失败后终端拿不到任何信息

BUG: CrossEntropy 的计算方法, 和 label 有冲突, 导致输出

,,,,, 的 的 的 的 ,,,,,,,,,,,,,, 的 的 的 的 的 的 的 。 [SEP] 。 [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP]
[SEP]
[SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP] [SEP]

这是因为 ignore_index 默认值会是 0, 而 0 正是我们的 label。所以必须需要把 ignore-index 调整到非 label 的值 (我选择了 -100)

思考: 为什么看起来更复杂、泛化能力更强的 dependency parse 还没有基于词性的 POS 准确性高?

我期望能学到: (错字, head_rel, head_word) → 正字, 这样的 pattern, 但在实验的过程中发

现了以下的问题。

把几个测试数据的树结构打印出来后，我发现一个错字、漏字或多字，就可能把整个短语的依存关系打乱，输出的 head/relation 本身就不准确，没法作为学习特征。同时纠错模型接受的是含错误的原始句子，dependency parser 在这种带错输入上往往会给出不可靠的树结构和关系标签。

同时，中文依存分析是以“词”为单位的，而我们做的是字符级纠错。为了把依存标签用到字符上，需要先做分词，之后再把每个词的 head/relation 赋值到每个字，这个过程中常常会造成对齐不准确（词边界错位、长词短词扩散不均），特征的噪声就更大。

思考：为什么 BiLSTM 在模型设计中不是必须的，CRF 反而是更重要的？

理论上 BERT 自身已经是深层双向 Transformer，具备丰富的上下文信息。额外的 BiLSTM 可能贡献有限，却增加了参数和过拟合风险。我觉得可以做出以下两个方面的改进：

一方面可以尝试直接使用 BERT 最后一层输出接分类头（类似于序列标注的 BERT+CRF 常见架构），减少模型复杂度；

另一方面，如果希望利用序列标注的标签依赖，可引入 CRF 层替代 BiLSTM，用于学习标签的全局约束（如避免无效的标签序列）。CRF 能够确保输出操作序列的一致性，但也会增加计算量。

综上，基于目前数据规模，优先考虑去除或简化 BiLSTM，让 BERT 的表示直接服务于分类，以减少过拟合。实验中也证明了这一点。